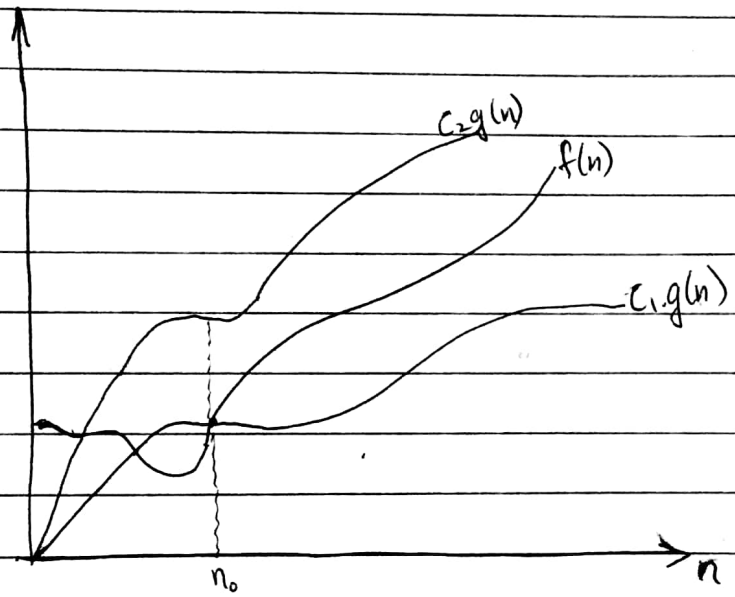
Date.....

① We are concerned with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.

② Θ-notations (asymptotic tight bound)

$$\Theta(g(n)) = \{ f(n): \text{there exist positive constants } c_1, c_2 \text{ \& } n_0 \text{ such that :-} \quad 0 \le c_1 \cdot g(n) \le f(n) \le c_2 g(n) \quad \text{for all } n \ge n_0. \}$$




- Here, for every value of $n \ge n_0$, the function $f(n)$ lies b/w $c_1 g(n)$ & $c_2 g(n)$.
- Here, $g(n)$ is tight bound for $f(n)$.
- The definitions of $\Theta(g(n))$ requires $f(n)$ to be asymptotically non-negative. i.e. $f(n)$ is non-negative whenever $n$ is sufficiently large. So $g(n)$ itself must be asymptotically non-negative, or else $\Theta(g(n))$ is empty. Therefore every function used within Θ-notation is non-negative asymptotically.

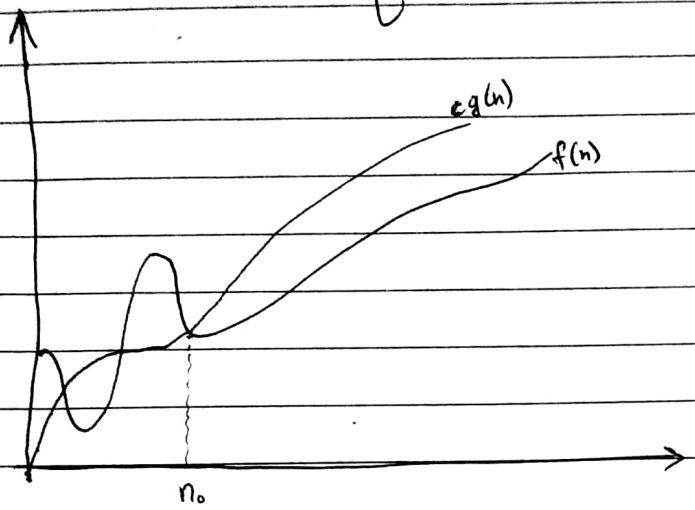
Spiral

→ asymptotic upper bound.

Date.....

③ O-notation:

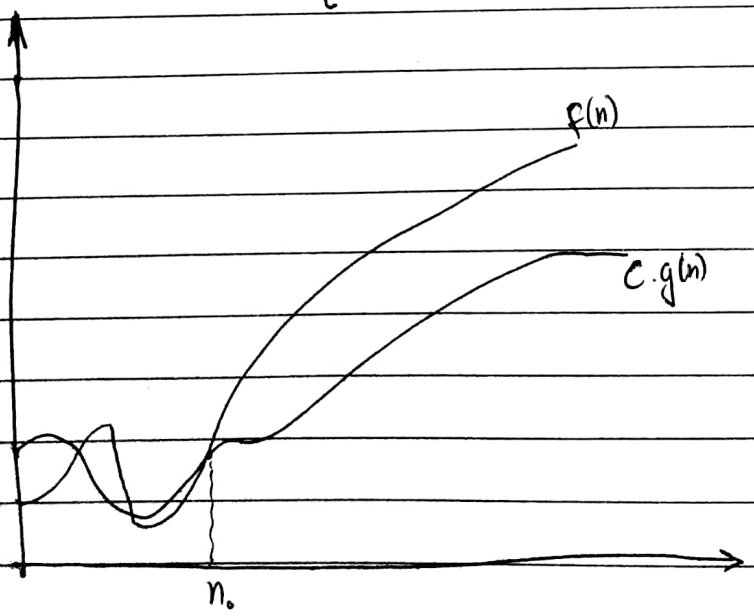
$$O(g(n)) = \{ f(n): \text{there exist positive constants } c \text{ and } n_0 : 0 \le f(n) \le cg(n), \text{ for all } n \ge n_0 \}$$




→ (asymptotic lower bound)

④ Ω-notation:

$$\Omega(g(n)) = \{ f(n) = \text{there exist positive constants } c \text{ and } n_0 : 0 \le c.g(n) \le f(n), \text{ for all } n \ge n_0 \}$$




Spiral

Date.....

⑤ o-notation:

$o(g(n)) : \{ f(n) \rightarrow$ there exist positive constants $c$ and $n_0$, $0 \leq f(n) < c\,g(n)$, for all $n \geq n_0 \}$

For ex: $[2n \in o(n^2)]$ but; $[2n^2 \notin o(n^2)]$.

⑥ ω-notation:

$\omega(g(n)) : \{ f(n)$, there exist positive constants $c$ and $n_0$, $0 \leq c\,g(n) < f(n)$, for all $n \geq n_0 \}$

For ex: • $2n^2 \in \omega(n^2)$ but $2n^2 \notin \omega(n^2)$.

⑦ Mathematical evaluation:

- $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \implies [f(n) = o(g(n))]$

- $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty \implies [f(n) = \omega(g(n))]$

Spiral

Date.....

## (8.) Recurrence Relations:

Whenver we tries to describe the recursion functions with the help of mathematical equations, they are called recurrence relations.

## (9.) Iteration method:

Date.....

## (10.) Substitution method:

The substitution method for solving recurrences comprises two steps:

① Guess the form of the solution.

② Use mathematical induction to find the constants and show that the solution works.

- This method is powerful, but we must be able to guess the form of the answer in order to apply it.
- As an example, lets establish an upper bound on the recurrence:—

$$T(n) = 2T(\lfloor n/2 \rfloor) + n.$$

→ we guess that the solution is $T(n) = O(n \log n)$. So we have to prove that:—

$[T(n) \leq cn.\log(n)]$, for appropriate choice of constant $c > 0$.

→ we can assume that this bound holds for all positive $m < n$ in particular for $m = \lfloor n/2 \rfloor$, giving:—

$$T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor . \lg(\lfloor n/2 \rfloor).$$

$$\begin{aligned} \therefore T(n) &\leq 2c.\lg(\lfloor n/2 \rfloor).\lfloor n/2 \rfloor + n. \\ &\leq 2cn.\log(n/2) + n \\ &= cn.\log(n) - cn.\log(2) + n \\ &= cn.\log(n) - cn + n. \\ &\leq nc.\log(n) \end{aligned}$$

And here last step holds as long as $c \geq 1$, but the boundary condition of $T(1) = 1$ fails there, so

Spiral

Date

we should say $C \geq 2$, we can give final proof as

$T(2) = 4$ and $T(2) \leq 2C \log(2)$

$T(2) \leq 4.\log(2)$.

which is correct. So,

$$\left[T(n) \leq cn.\log(n) \quad (\text{for } \begin{matrix} c \geq 2. \\ n_0 = 2. \end{matrix})\right]$$

(11) Ex: for changing variables:-

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n.$$

∴for convinince, we shall not worry about rounding off values, such as $\sqrt{n}$ to integers.

let, $(m = \log_2 n) \longrightarrow n = 2^m$

$$\left[T(2^m) = 2T(2^{m/2}) + m.\right]$$

now, we can rename $\Rightarrow S(m) = T(2^m)$

So; $\left[S(m) = 2.S(m/2) + m\right]$

↳ we know its solution as $\left[S(m) = O(m.\log m)\right]$

or, $T(2^m) = O(\log(n).\log(\log(n)))$

$T(n) = O\left[\log(n).\log(\log(n))\right]$.

Spiral

Date.....

## (12) The recursion tree method:

- In recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- We sum the costs within each level of the tree to obtain a set of per-level cost.
- Then we sum all the per-level costs to determine the total cost of all levels of recursion.

## (13) Master method:-

It is a type of cookbook to solve recurrence relations:-

$$T(n) = aT\left(n/b\right) + f(n).$$

Here, $a \geq 1$ & $b > 1$, are const. and $f(n)$ is an asymptotically positive function.
And, ($n/b$ is mean to either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$.)

There,

① If $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for some constant $\varepsilon > 0$, then

$$\boxed{T(n) = \theta\left(n^{\log_b a}\right).}$$

② If $f(n) = \theta\left(n^{\log_b a}\right)$, then $\boxed{T(n) = \theta\left(n^{\log_b a} \cdot \lg n\right) \cong \theta\left(f(n) \cdot \lg n\right)}$

Date.....

③ If $f(n) = \Omega\left(n^{\log_b a+\varepsilon}\right)$ for some const. $\varepsilon > 0$,

and if $\left[af(n/b) \leq c.f(n)\right]$ for some const. $c < 1$
and all sufficiently large $n$, then $\left[T(n) = \Theta(f(n))\right]$.

→ This is called regularity condition.

Eg. $T(n) = 9T(n/3) + n.$

$a = 9,\ b = 3 \rightarrow n^{\log_b a} \Rightarrow n^2$

So, $f(n) = O\left(n^{\log_3 9 - \varepsilon}\right)$ where $\varepsilon = 1$.

First case applies → $\left[T(n) = \Theta(n^2)\right]$

Date.....

## (14) Master theorem for subtract and conquer Recurrence:-

~~It is used to determine the Big-O upper bound on functions:~~

let $T(n)$ be a function defined on positive $n$ as

$$T(n) \leq \begin{cases} c ; & \text{if } n \leq 1 \\ a.T(n-b) + f(n) ; & \text{if } n > 1 \end{cases}$$

for some constants $c, a>0, b>0, k \geq 0$. ~~and function~~

- If $f(n)$ is in $O(n^k)$, then;

$$T(n) = \begin{cases} O(n^k) & , \text{if } a<1 \\ O(n^{k+1}) & , \text{if } a=1 \\ O(n^k . a^{n/b}) & , \text{if } a>1 \end{cases}$$

### Proof by substitution:

- from above function we have:

$$T(n) = a.T(n-b) + f(n).$$
$$T(n-b) = a.T(n-2b) + f(n-b).$$
$$T(n-2b) = a.T(n-3b) + f(n-2b)$$

Date.....

Now;

$$T(n-b) = a^2 . T(n-3b) + a . f(n-2b) + f(n-b)$$

$$T(n) = a^3 . T(n-3b) + a^2 . f(n-2b) + a . f(n-b) + f(n)$$

or;

$$\boxed{T(n) = \sum_{i=0}^{n} . a^i . f(n-ib) + C}$$

Here, $f(n-ib) = O(n-ib)$.

$$T(n) = O\left(n^k . \sum_{i=0}^{n/b} a^i\right)$$

where, if $a<1$, then $\sum_{i=0}^{n/b} a^i = O(1) \Rightarrow T(n) = O(n^k)$.

if $a=1$, then $\sum_{i=0}^{n/b} a^i = O(n) \Rightarrow T(n) = O(n^{k+1})$.

if $a>1$, then $\sum_{i=0}^{n/b} a^i = O(a^{n/b}) \Rightarrow T(n) = O(n^k . a^{n/b})$.

Spiral

Date.....

# Data Structure for disjoint sets:-

→ Some application involve grouping of elements into a collection of disjoint sets. The disjoint sets are those sets which have no element in common.

→ A disjoint set data structure maintains a collection;

$$'S' = \{ S_1, S_2 \text{ --- } S_k \} \text{ of disjoint sets.}$$

→ Each set is identified by a representative which is some member of the set. There could be some rule for choosing one, such as choosing the smallest member in the set etc.

• Let, x denotes the object or member of a set we wish to support the following operations:

① MAKE_SET(x):
It creates a new set whose only member is x. Since, the sets are disjoint, we require that x is not already in some other set.

② UNION (x,y);
Unites the dynamic sets that contains x & y, say $S_x$ and $S_y$ into a new set that is union of these two sets.

Note: $S_x$ & $S_y$ assumed to be disjoint prior to UNION operation & thus they are removed from 'S' after their UNION.

Date.....

③ FIND_SET (x):

returns a pointer to the representative of the set containing x.

- We analyze the running time of disjoint-set data structure in terms of 2 parameters:—
  - → n ⟹ no. of MAKE_SET operations.
  - → m ⟹ no. of MAKE_SET, UNION, FIND operations.

- Since the sets are disjoints, each union operation reduces the number of sets by one. With (n-1) union operations, we are left with only one set.

- Also to form 'n' sets, n MAKE_SET operations and as 'm' includes MAKE_SET, UNION and FIND operations,

$$m \geq n$$

(more no. of operations in m, than n.).

## Application:

① ~~Determining the~~ connected components of an undirected graph:—

```
CONNECTED_COMPONENTS (G)
    for each vertex v ∈ V[G]
        do MAKE_SET(v)
    for each edge (u,v) ∈ E[G]
        do if FIND_SET (u) ≠ FIND_SET (v)
            then UNION (u,v).
```

Note:

- V[G]: set of vertices.
- E[G]: set of edges.

Spiral

Date.....

## ② SAME_COMPONENTS:-

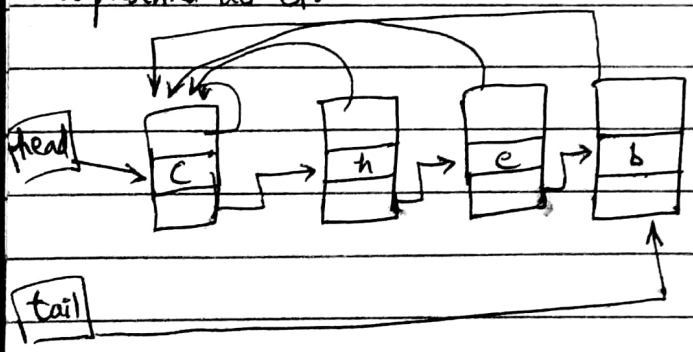
answers the queries about whether two vertices are in the same connected component.

```
SAME_COMPONENT (u,v)
    if FIND_SET(u) == FIND_SET(v)
            RETURN TRUE
    else
            RETURN FALSE
```

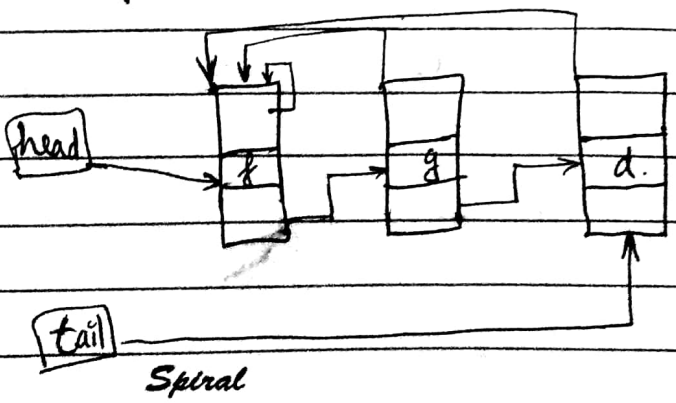
## # Linked-List REPRESENTATION OF DISJOINT SETS:-

$S_1 = \{c, h, e, b\}$ $S_2 = \{f, g, d\}$

- Represented as $S_1$:




- Representation of $S_2$:




Spiral

Date.....

- ① first object in linked list serves as its set's representative.

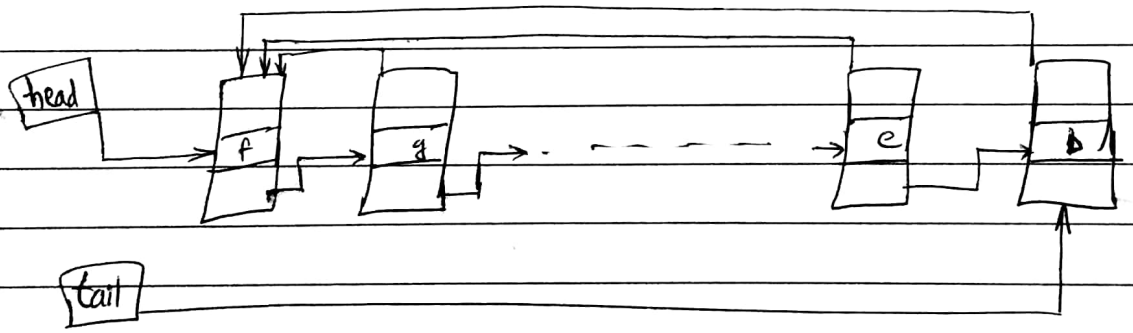
② Each object contains a set member, a pointer to the next set member and a pointer back to the representative.

③ Head points to representative and tail points to last object in the set.

- ① MAKE-SET(x): Require O(1) time, create a new link list whose only member is x.

② FIND-SET(x): Require O(1) time, as return the pointer from x back to representative.

③ Simple implementation of UNION:




→ we perform UNION(x,y) by appending 'x' list on the end of 'y' list.

→ we use tail pointer of y's set to find where to append x's list.

→ Representative is the representative of set containing y.

→ we update the pointer for each object originally on x's list, to point back to head of y's list.

→ Forest heuristics.

Date.....

# PATH compression:-

very simple and effective. We use path compression during FIND_SET operations to make each node find path points directly to the root.

→ Path compression does not change any rank.

☆ We perform three operations:-

① MAKE_SET : simply creates a tree.

② UNION : causes the root of one tree to point to the root of other.

③ FIND_SET: by following parents pointers until, we find root of the tree.

- MAKE_SET (x)
  - ① x.p = x.
  - ② x.rank = 0.

- UNION (x, y)
  - ① LINK (FIND_SET(x), FIND_SET (y))

- LINK (x, y)
  - ① if x.rank > y.rank
  - ② y.p = x.
  - ③ else x.p = y.
  - ④ if x.rank == y.rank
  - ⑤ y.rank = y.rank + 1.

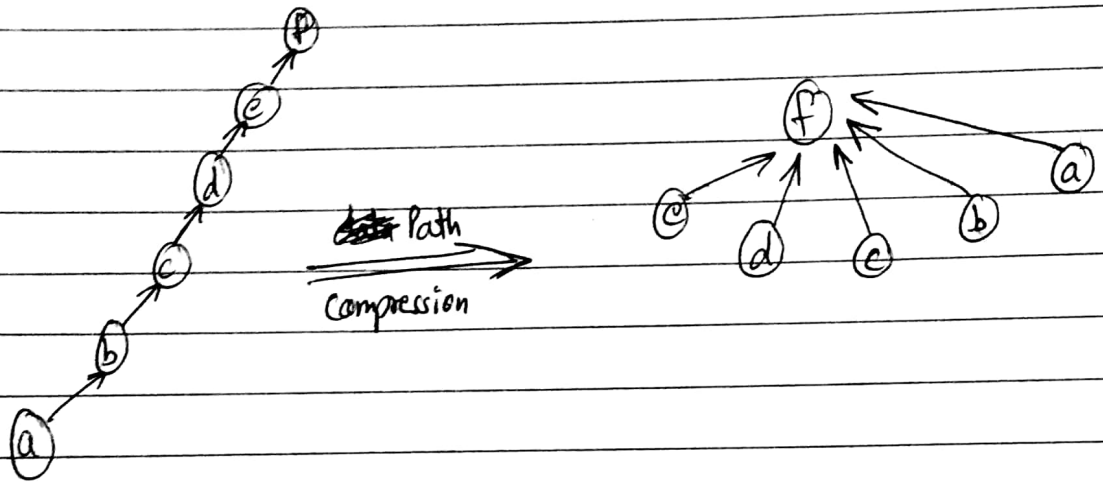
Spiral

Date.....

∴ The FIND_SET procedure with path compression:-

- FIND_SET (x)
  if $x \neq x.p$
    $x.p = FIND\_SET(x.p)$
  return $x.p$.

Eg:




- After executing FIND_SET(a), each node on the find path points to the root node. It will take linear time.

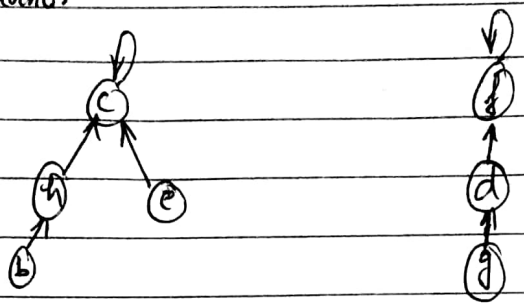
Date.....

# Disjoint Forest:

For faster implementation, we represent sets by rooted trees, with each node containing one member and each tree representing one set. Moreover, each member points to its parent.

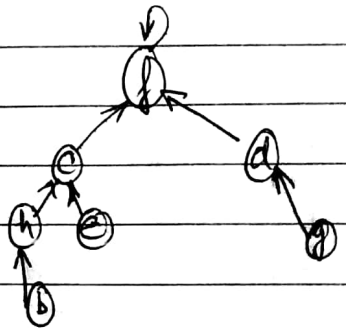
Eg.

$S_1 = \{c, h, e, b\}$.




• union of these two sets →

Here, root of one tree to point to the root of the other




# Heuristic: To improve the running time. We can achieve a running time better than linked list representation. By using two heuristics, we can achieve a running time that is almost linear in the total number of operations m.

① Union by rank: The approach would be to make the root of the tree with fewer nodes point to the root of the tree with more nodes.

For each node, we maintain a rank, which is an upper bound on the height of the node. During union by rank, we make the root with smaller rank point to root with larger rank point.

Date.....

@ Path compression:
(Same as earlier.)

Note: representation by linked list has only union heuristic while the representation by forest, has union as well as path compression heuristics.

# Mean and order Statistics:—

- $K^{th}$ order statics of a set of n elements is the $K^{th}$ smallest element.
  if $k=1 \Longrightarrow$ minimum element
  if $k=n \Longrightarrow$ max elemenent.

- Median: $\lfloor \frac{n+1}{2} \rfloor$ or $\lceil \frac{n+1}{2} \rceil$
  lower median — upper median.

- for finding order statistics of $K^{th}$ element,
  → (i) Randomized algo.
  → (ii) Wost case linear time.

Spiral

Date.....

- Randomized algo →

- RANDOMISED_SELECT (A, P, q, i)

```
if (p==q)
    then return A[P]

r = RANDOMPARTITION (A, P, q).

K = r - P+1                    //k is the rank of r in A.

if (i=k)
    then return A[r].

elseif (i<k)
    then return Randomized-select (A, P, r-1, i)

else
    return Randomized-select (A, r+1, q, q-k)
```

The will work will an array A, p starting index, q is last index and i is the $i^{th}$ static.

- Secondly, we could use merge sort, to sort the array and then giving order statics., with time O(n.logn)

Spiral

$O(\lg n)$

Date.....

- While in case of tree, with root x;

```
• OS_SELECT (x,i)
        r = x.left.size +1

        if i==r
            return x.

        elseif i<r
            return OS_SELECT (x.left , i)

        else
            return  OS_SELECT (x.right, i-r).
```

- in order static tree in linear time:- (given a node, find its position)

```
• OS_RANK (T,x).
        r=x.left.size+1
        y=x.

        while y ≠ T.root
            if y == y.p.right
                r=r+ y.p.left.size +1
            y=y.p

        return r.
```

$O(\lg(n))$

Date.....

# Insertion sort:- $\Theta(n^2)$ $O(1)$

- It takes max time if elements are sorted in reverse order. and min time when they are already sorted.
- It is used when number of elements is small.

- INSERTION_SORT (A)

```
for i=2 to n
    key ← A[i]
    j ← i-1

    while j>0 and A[j]>key
        A[j+1] ← A[j]
        j ← j-1

    A[j+1] ← key
```

worst case: $\Theta(n^2)$

best case: $\Theta(n)$

# Selection sort: $O(n^2)$ $O(1)$

- The algo maintains two subarrays in a given array:
  - The subarray which is already sorted.
  - Remaining subarray which is unsorted.

Spiral

for all three

Date.....

# Merge sort: $\Theta(n \log n)$ $O(n)$

- It divides input array in two halves, call itself for the two halves, ~~calls~~ and then merges the two sorted halves.

- Merge_SORT (arr[], l, r)
  if r > l
  ① Find the middle ~~point~~ point to divide the array
  middle ~~m~~ = (l+r)/2.

  ② call merge sort for first half.
  call merge_SORT (arr, l, m)

  ③ call merge sort for second half.
  call merge_sort (arr, m+1, r).

  ④ Merge the two halves sorted in step 2 and 3;
  call merge (arr, l, m, r)

  else
  return null;

- It is useful in sorting a linked list.
- It is divide and conquer algo.

Date.....

# Quick Sort: $\theta(n^2)$

- It picks an element as pivot and partitions the given array around the picked pivot. And there are several ways to select a pivot point.

```
• Quick_Sort (arr[], low, high).
{
        if (low < high)
        {
            pi = partition (arr, low, high)   // it is partiting index.

            quick_SORT (arr, low, pi-1);
            quick_SORT (arr, pi+1, high);
        }
}

Partition (arr[], low, high)
{       pivot = arr[high]

        i = low - 1 ;          // index of smaller element.

        for (j=low ; j ≤ high -1 ; j++)
        {       if (arr[j] < pivot)
                {       i++ ;
                        swap (arr[i], arr[j])
                }
        }
        swap (arr[i+1] , arr[high])
        return i+1;
}
```

Spiral

Date.....

- For worst case → $\theta(n^2)$
  best case → $\theta(n \log n)$
  average case → $\theta(n \log n)$

# Strassen's algorithm:- $O(n^{\log_2 7})$

- Normal matrix multiplication procedure takes $\theta(n^3)$ time.

## Simple divide and conquer algo:- (just to read)

- Square_Matrix_Multiply_Recursive (A, B)

  n = A.rows

  let C be a new n×n matrix

  if n == 1

  $c_{11} = a_{11} \cdot b_{11}$

  else partition A, and B and C as in equations:-

  $C_{11} = SMMR(A_{11}, B_{11}) + SMMR(A_{12}, B_{21})$;

  $C_{12} = SMMR(A_{11}, B_{12}) + SMMR(A_{12}, B_{22})$;

  $C_{21} = SMMR(A_{21}, B_{11}) + SMMR(A_{22}, B_{21})$;

  $C_{22} = SMMR(A_{21}, B_{12}) + SMMR(A_{22}, B_{22})$;

  return C.

→ It is also → $\theta(n^3)$.

Spiral

Date.....

## ★ Strassen's Method:

- Here instead of performing eight recursive multiplication of $n/2 \times n/2$ matrices, we performs only seven.
- The cost of eliminating one matrix multiplication will be several new additions of $n/2 \times n/2$ matrices, but still only a constant number of additions.

① Divide the input matrix A, B and output matrix C into $n/2 \times n/2$ submatrices.

② Create 10 matrices $S_1, S_2 \dots S_{10}$ each of which $n/2 \times n/2$ and is sum or difference of two matrices created in step 1.

③ Using the submatrices of ② & ①, recursively compute seven matrix products $P_1, P_2 \dots P_7$. Each matrix $P_i$ is $n/2 \times n/2$

④ Compute the desired submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ of the result matrix C, by using $P_i$ matrices.

- $$T(n) = \begin{cases} \theta(1) & \text{if } n=1 \\ 7T(n/2) + \theta(n^2) & \text{if } n>1 \end{cases}$$

By master method $\Rightarrow$ $\boxed{T(n) = \theta(n^{\log 7})}$

Date.....

- $S_1 = B_{12} - B_{22}$
  $S_2 = A_{11} + A_{12}$
  $S_3 = A_{21} + A_{22}$
  $S_4 = B_{21} - B_{11}$
  $S_5 = A_{11} + A_{22}$
  $S_6 = B_{11} + B_{22}$
  $S_7 = A_{12} - A_{22}$
  $S_8 = B_{21} + B_{22}$
  $S_9 = A_{11} - A_{21}$
  $S_{10} = B_{11} + B_{12}$

- $P_1 = A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22}$

  $P_2 = S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22}$

  $P_3 = S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11}$

  $P_4 = A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11}$

  $P_5 = S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}$

  $P_6 = S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}$

  $P_7 = S_9 S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}$

- $C_{11} = P_5 + P_4 - P_2 + P_6$

  $C_{12} = P_1 + P_2$

  $C_{21} = P_4 + P_3$

  $C_{22} = P_5 + P_1 - P_3 - P_7$